Back to our contacts application, how would we use a Bloom filter to make it faster? Well, as mentioned, we need to use it as a dictionary, so we are going to create a new Bloom filter when our email application starts, retrieve all the contacts from the server, and add them to the Bloom filter: listing 4.1 summarizes this initialization process.

Listing 4.1 Starting up email application

```
function initBloomFilter(server,
minSize)                                        #1
  contactsList ←
server.loadContacts()
#2
  size ← max(2 * |contactsList|,
minSize)                                        #3
  bloomFilter ← new
BloomFilter(size)                                       #4
  for contact in contacs
do                                                  #5

bloomFilter.insert(contact)
            #6
  return bloomFilter
```

#1 Method `initBloomFilter` takes an interface to a server (a *façade* object) and the minimum size that should be used to initialize the Bloom filter; it returns the newly created Bloom filter.

#2 On startup, optionally loads the list of contacts from a server that takes care of durable storage.

#3 The size of the Bloom filter should be at least twice the current contact list's, but at least equal to `minSize`, a minimum value that can be passed as an argument.